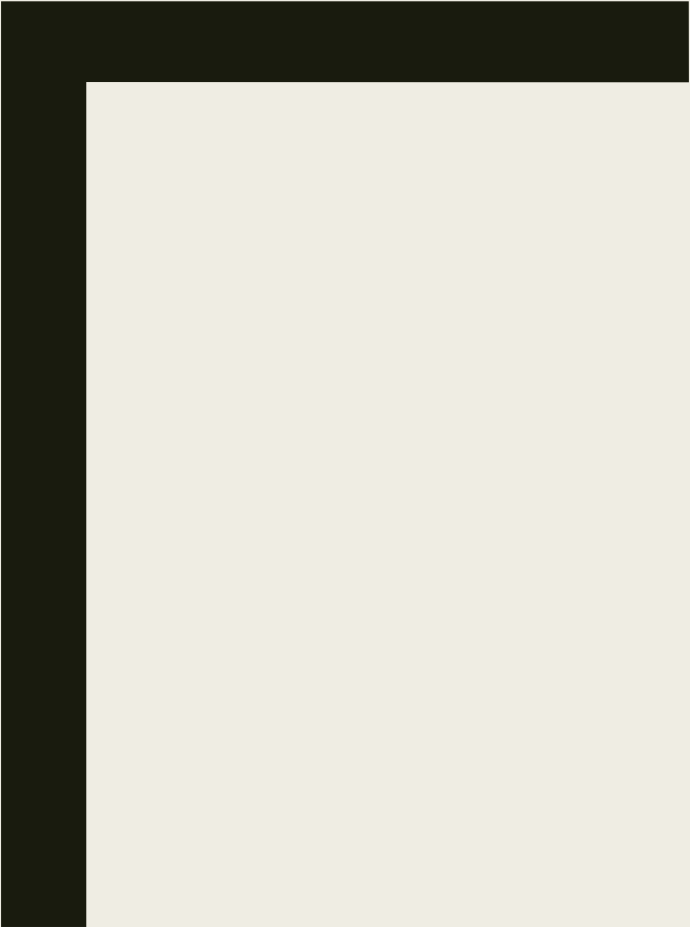




LISTAS

Estructura de Python



Listas en Python

- Las listas en Python son un tipo de dato que permite almacenar datos de cualquier tipo. Son mutables y dinámicas, lo cual es la principal diferencia con los sets y las tuplas. Una lista se crea con [] separando sus elementos con comas. Una gran ventaja es que pueden almacenar tipos de datos distintos.

```
lista = [1, 2, 3, 4]
```

```
lista = list("1234")
```

```
lista = [1, "Hola", 3.67, [1, 2, 3]]
```

Crear listas en Python

- Las listas en Python son uno de los tipos o estructuras de datos más versátiles del lenguaje, ya que permiten almacenar un conjunto arbitrario de datos. Es decir, podemos guardar en ellas prácticamente lo que sea. Si vienes de otros lenguajes de programación, se podría decir que son similares a los arrays.

Propiedades

- Son **ordenadas**, mantienen el orden en el que han sido definidas
- Pueden ser formadas por tipos **arbitrarios**
- Pueden ser **indexadas** con [i].
- Se pueden **anidar**, es decir, meter una dentro de la otra.
- Son **mutables**, ya que sus elementos pueden ser modificados.
- Son **dinámicas**, ya que se pueden añadir o eliminar elementos.

Métodos

`append(<obj>)`

El método `append()` añade un elemento al final de la lista.

```
l = [1, 2]
l.append(3)
print(l) #[1, 2, 3]
```

`extend(<iterable>)`

El método `extend()` permite añadir una lista a la lista inicial.

```
l = [1, 2]
l.extend([3, 4])
print(l) #[1, 2, 3, 4]
```

`insert(<index>, <obj>)`

El método `insert()` añade un elemento en una posición o índice determinado.

```
l = [1, 3]
l.insert(1, 2)
print(l) #[1, 2, 3]
```

`remove(<obj>)`

El método `remove()` recibe como argumento un objeto y lo borra de la lista.

```
l = [1, 2, 3]
l.remove(3)
print(l) #[1, 2]
```

`pop(index=-1)`

El método `pop()` elimina por defecto el último elemento de la lista, pero si se pasa como parámetro un `índice` permite borrar elementos diferentes al último.

```
l = [1, 2, 3]
l.pop()
print(l) #[1, 2]
```

`reverse()`

El método `reverse()` invierte el orden de la lista.

```
l = [1, 2, 3]
l.reverse()
print(l) #[3, 2, 1]
```

`sort()`

El método `sort()` ordena los elementos de menos a mayor por defecto.

```
l = [3, 1, 2]
l.sort()
print(l) #[1, 2, 3]
```

Y también permite ordenar de mayor a menor si se pasa como parámetro `reverse=True`.

```
l = [3, 1, 2]
l.sort(reverse=True)
print(l) #[3, 2, 1]
```

`index(<obj>[,index])`

El método `index()` recibe como parámetro un objeto y devuelve el índice de su primera aparición. Como hemos visto en otras ocasiones, el índice del primer elemento es el `0`.

```
l = ["Periphery", "Intervals", "Monuments"]
print(l.index("Intervals"))
```



También permite introducir un parámetro opcional que representa el índice desde el que comenzar la búsqueda del objeto. Es como si ignorara todo lo que hay antes de ese índice para la búsqueda, en este caso el `4`.

```
l = [1, 1, 1, 1, 2, 1, 4, 5]
print(l.index(1, 4)) #5
```